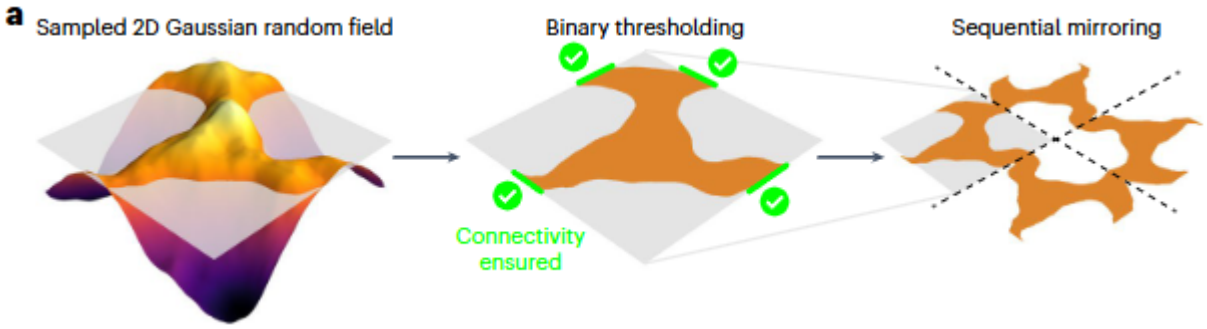
전형국 교수님 데이터셋 생성 보고서

논문 상 데이터 셋 생성 원리



Algorithm 1 Metamaterial unit cell generation

Set even pixel number N
while generated structure not valid **do**
  Sample complex $z_{k_1,k_2} \sim \mathcal{N}(0,1) + i\mathcal{N}(0,1)$ on centered $N \times N$ grid
  Compute $F_{k_1,k_2} = z_{k_1,k_2} P(k_1,k_2)$,
    where $P(k_1,k_2) \propto (k_1^2 + k_2^2)^{-\alpha/2}$ with $\alpha = 3$
  Transform to $N \times N$ pixel grid via inverse DFT as
    $f_{x_1,x_2} = \frac{1}{N^2} \sum_{k_1=-N/2}^{N/2-1} \sum_{k_2=-N/2}^{N/2-1} F_{k_1,k_2} e^{i\frac{2\pi}{N}x_1k_1} e^{i\frac{2\pi}{N}x_2k_2}$
  Compute real part standardized over all pixels as[1]
    $\bar{f}_{x_1,x_2} = \frac{\mathrm{Re}(f_{x_1,x_2}) - \mu[\mathrm{Re}(f_{\mathcal{X}})]}{\sigma[\mathrm{Re}(f_{\mathcal{X}})]}$
  Sample threshold $t \sim \mathcal{U}(0, t_{\max})$ with $t_{max} = 3/5$
  for $x_1 = 1$ to N **do**
    for $x_2 = 1$ to N **do**
      if $\bar{f}_{x_1,x_2} > t$ **then**
        $\bar{f}_{x_1,x_2} \leftarrow 1$
      else
        $\bar{f}_{x_1,x_2} \leftarrow 0$
      end if
    end for
  end for
  if $\bar{f}$ is four-side connected with at least 10% pixel-coverage **then**
    Mirror $\bar{f}$ sequentially along vertical and horizontal border
    return $\bar{f}$
  end if
end while

알고리즘 순차적 설명

1단계: 초기 설정

- **픽셀 수 설정**: `N x N` 크기의 그리드를 만듭니다. 이때 `N`은 짝수입니다.
- **반복 시작**: 유효한 구조가 생성될 때까지 이후의 모든 과정을 반복하는 루프(`while`)를 시작합니다.

2단계: 무작위 주파수 패턴 생성

- **복소수 샘플링**: `N x N` 그리드의 각 지점에서 무작위로 복소수(`z`)를 생성합니다. 이 복소수의 실수부와 허수부는 모두 평균이 0이고 표준편차가 1인 정규분포를 따릅니다. 이는 주파수 공간에서 무작위 패턴의 기초가 됩니다.
- **저역 통과 필터 적용**: 생성된 복소수 `z`에 `P`라는 필터를 곱하여 `F`를 계산합니다. 이 필터는 주파수가 낮은 중심부의 값은 강조하고, 주파수가 높은 외곽부의 값은 약화시키는 역할을 합니다.

이를 통해 최종 구조가 너무 급격하게 변하지 않고 부드러운 형태를 갖도록 유도합니다.

```
def gaussian_random_field(alpha = 1.0,
                          size = 128,
                          seed = None,
                          flag_normalize = True):
    if seed:
        np.random.seed(seed=seed)
    k_ind = np.mgrid[:size, :size] - int( (size + 1)/2 )
    k_idx = scipy.fftpack.fftshift(k_ind)
    amplitude = np.power( k_idx[0]**2 + k_idx[1]**2 + 1e-10, -alpha/4.0 )
    amplitude[0,0] = 0
    noise = np.random.normal(size = (size, size)) \
        + 1j * np.random.normal(size = (size, size))
    gfield = np.fft.ifft2(noise * amplitude).real
    if flag_normalize:
        gfield = gfield - np.mean(gfield)
        gfield = gfield/np.std(gfield)
    return gfield
```

3단계: 공간 패턴으로 변환

- **역 푸리에 변환(Inverse DFT)**: 주파수 공간의 데이터 `F`를 역 푸리에 변환하여 실제 공간의 `N x N` 픽셀 그리드 `f`로 변환합니다. 이 과정을 통해 알아볼 수 있는 이미지 형태의 패턴이 만들어집니다.
- **데이터 표준화**: 변환된 그리드 `f`의 실수(Real part) 값들만 사용합니다. 이 값들의 전체 평균과 표준편차를 구해, 각 픽셀 값을 `(원래 값 - 평균) / 표준편차` 공식으로 **표준화**합니다. 이 결과로 나온 새로운 그리드가 $\bar{f}$입니다.

4단계: 흑백 이미지로 변환 (이진화)

- **임계값 설정**: 0과 3/5 사이에서 무작위로 임계값 `t`를 선택합니다.
- **픽셀 값 결정**: $\bar{f}$ 그리드의 모든 픽셀을 확인합니다.
  - 픽셀 값이 임계값 `t`보다 **크면** 픽셀 값을 **1** (흰색)로 설정합니다.
  - 픽셀 값이 임계값 `t`보다 **작거나 같으면 0** (검은색)으로 설정합니다.
  - 이 과정을 통해 모든 픽셀이 0 또는 1의 값을 갖는 흑백 이미지가 완성됩니다.

5단계: 유효성 검사 및 최종 완성

- **조건 확인**: 생성된 흑백 구조 $\bar{f}$가 다음 두 가지 조건을 만족하는지 검사합니다.
  1. **사방 연결 (Four-side connected)**: 구조의 상하좌우 경계가 모두 끊어지지 않고 이어져 있는가?
  2. **최소 면적 (10% pixel-coverage)**: 전체 픽셀 중 1(흰색)이 차지하는 비율이 10% 이상인가?
- **결정**:
  - **조건 만족 시**: 구조를 상하, 좌우로 대칭 복사(미러링)하여 최종 단위 셀을 완성하고 알고리즘을 종료합니다.

- **조건 불만족 시**: 생성된 구조를 버리고, **2단계(무작위 주파수 패턴 생성)부터** 모든 과정을 다시 반복합니다.

```python
def generate_geometry(grf_alpha, pixels, pixel_threshold_rel, grf_threshold_rel):

    pixel_threshold = int(pixels*pixel_threshold_rel)
    grf_threshold = rand.uniform() * grf_threshold_rel

    flag = False
    while flag == False:
        filling_bin = gaussian_random_field(alpha = grf_alpha, size = pixels).reshape(-1)
        filling_bin[filling_bin < grf_threshold] = 0
        filling_bin[filling_bin > grf_threshold] = 1
        geom_flattened = filling_bin.copy().astype('int')
        geom_l = geom_flattened[0:-1:pixels]
        geom_r = geom_flattened[pixels-1::pixels]
        geom_d = geom_flattened[-pixels:]
        geom_u = geom_flattened[0:pixels]

        geom = geom_flattened.reshape(pixels,pixels)

        if (np.sum(geom_l) >= pixel_threshold and np.sum(geom_r) >= pixel_threshold
            and np.sum(geom_d) >= pixel_threshold and np.sum(geom_u) >= pixel_threshold):
            G = create_graph(geom)
            if nx.is_connected(G):
                flag = True

    return geom_flattened
```

Abaqus 코드 내에서 Geometry를 part로 변형

```
def create_part(geom, pixels, radius=0):

    elements_per_pixel = 2 # must at least be 2

    elements = pixels*elements_per_pixel

메시(Mesh) 세분화: 입력된 픽셀보다 더 정밀한 그리드(격자)를 설정합니다.
`elements_per_pixel`은 하나의 픽셀을 몇 개의 작은 요소로 나눌지를 결정합니다.


    nx, ny = (elements+1, elements+1)

    x = np.linspace(0, 1, nx)

    y = np.linspace(0, 1, ny)

    xv, yv = np.meshgrid(x, y)

    coords = np.stack((xv.flatten(),yv.flatten()),axis=1)

  좌표 및 노드 생성: `linspace`와 `meshgrid`를 사용해 0과 1 사이의 정규화된 2D 공
```

간에 정밀한 그리드 좌표(`coords`)를 생성합니다. 이 좌표 하나하나가 형상을 구성하는 노드(node)가 됩니다.

```
    full_edges = np.zeros((2*elements*(elements+1),2), dtype=int)
```

전체 엣지(Edge) 생성: 이 그리드에 존재할 수 있는 모든 수평, 수직 엣지(edge, 모서리) 목록(`full_edges`)을 미리 만들어 둡니다. 엣지는 두 개의 노드를 연결하는 선입니다.

```
    temp = 0

    for i in range(elements+1):

        for j in range(elements):

            full_edges[temp,:] = [i*(elements+1)+j,i*(elements+1)+j+1]

            temp += 1

    for i in range(elements+1):

        for j in range(elements):

            full_edges[temp,:] = [j*(elements+1)+i,(j+1)*(elements+1)+i]

            temp += 1



    # if elements_per_pixel > 1:

    geom =
np.repeat(np.repeat(geom.reshape(pixels,pixels),elements_per_pixel,axis=1),e
lements_per_pixel,axis=0)
```

픽셀 형상 확대: 입력받은 `geom` (예: 64x64 픽셀)을 앞에서 만든 정밀 그리드 크기에 맞게 확대(`np.repeat`)합니다.

```
    diff_x = np.diff(geom,axis=0)

    diff_y = np.diff(geom,axis=1)



    full_diff_x_rev = np.concatenate((geom[0,:][None],diff_x,geom[-1,:]
[None]),axis=0)

    full_diff_x = full_diff_x_rev[::-1,:]

    full_diff_y_rev = np.concatenate((geom[:,0][:,None],diff_y,geom[:,-1]
```

```
[:,None]),axis=1)

    full_diff_y = full_diff_y_rev[::-1,:]



    true_edges_bin =
np.abs(np.concatenate((full_diff_x.flatten(),np.transpose(full_diff_y).flatt
en()),axis=0))

    true_edges_bin_idx = np.argwhere(true_edges_bin==True).flatten()



    true_edges = full_edges[true_edges_bin_idx]
```

경계선 찾기: `np.diff`를 사용해 확대된 형상에서 인접한 픽셀 값의 차이를 계산합니다. 픽셀 값이 0(빈 공간)에서 1(재료)로, 또는 1에서 0으로 바뀌는 지점이 바로 형상의 경계선(엣지)이 됩니다.

```
    full_permutations = []
```

연속된 경로 찾기: `true_edges` 목록에 있는 엣지들을 순서대로 연결하여 형상의 완전한 외곽선(들)을 찾아냅니다. 이 과정은 `while` 루프를 통해 이루어집니다.
루프 동작 방식:

1. 남아있는 엣지 중 하나를 시작점으로 선택합니다.

2. 그 엣지의 끝점과 연결된 다른 엣지를 찾습니다.

3. 찾은 엣지를 경로에 추가하고, 목록에서 제거합니다.

4. 더 이상 연결할 엣지가 없을 때까지 2~3번 과정을 반복하여 하나의 닫힌 경로(외곽선)를 완성합니다.

5. 이 경로를 `full_permutations` 리스트에 저장합니다.

6. 만약 내부에 구멍이 있거나 분리된 형상이 있다면, 아직 처리되지 않은 엣지들이 남아있으므로 루프가 반복되어 다른 경로들도 찾아냅니다.

```
    while true_edges.shape[0] != 0:

        flag1 = True

        flag2 = True

        cur_permutation = []

        while flag2:
```

```
            if flag1 == True:
                cur_permutation.append(true_edges[0,0])
                cur_permutation.append(true_edges[0,1])
                cur_val = true_edges[0,1]
                true_edges = np.delete(true_edges, 0, axis=0)
                flag1 = False
            idx1 = np.where(true_edges == cur_val)[0][0]
            idx2 = np.where(true_edges == cur_val)[1][0]
            cur_permutation.append(true_edges[idx1,idx2])
            cur_permutation.append(true_edges[idx1,1-idx2])
            cur_val = true_edges[idx1,1-idx2]
            true_edges = np.delete(true_edges, idx1, axis=0)
            if len(np.where(true_edges == cur_val)[0]) == 0:
                flag2 = False
        full_permutations.append(cur_permutation)
        flag2 = True
        flag1 = True

    red_permutations = []

    for list in full_permutations:
        red_list = []
        for iter, i in enumerate(list):
            if iter == 0 or iter == len(list)-1:
                cur_pos = coords[i]
```

```
                temp = i

                red_list.append(temp)

                continue

            else:

                if ((not np.isclose(coords[i,0], cur_pos[0]))

                and (not np.isclose(coords[i,1], cur_pos[1]))):

                    cur_pos = coords[i]

                    red_list.append(temp)

                temp = i

        red_permutations.append(red_list)
```

불필요한 점 제거: 3단계에서 찾은 외곽선 경로에는 직선 구간의 모든 노드 정보가 포함되어 있습니다. 이 단계에서는 직선 구간의 중간 점들을 제거하고 코너(corner) 지점의 노드들만 남겨 경로를 단순화합니다.

```
    m.ConstrainedSketch(name='__profile__', sheetSize=2.0)
```

스케치 환경 시작: `m.ConstrainedSketch`를 호출하여 CAD 소프트웨어 내에 `__profile__`이라는 이름의 2D 스케치 평면을 생성합니다.
코너 노드들의 경로는 `red_permutations`에 저장

```
    full_surf_list = []

    top_surf_list = []

    bot_surf_list = []

    outer_surf_no_boundary_list = []

    inner_node_list = []



    longest_permutation = 0



    for idx, list in enumerate(red_permutations):
```

외곽선 그리기: `red_permutations`에 저장된 각 외곽선 경로를 순회하며 스케치 평면에 그림을 그립니다.

- 직선 모서리 (`radius=0`): `Line` 명령어를 사용해 코너 점들을 직선으로 연결합니다.

- 곡선 모서리 (`radius>0`): 직선의 일부를 잘라내고 그 자리에 `ArcByCenterEnds` 명령어로 부드러운 호(arc)를 그려 넣어 모서리를 둥글게 만듭니다.

- 스플라인 (`splined=True`): `Spline` 명령어를 사용해 점들을 부드러운 곡선으로 연결합니다.

경계면 정보 저장: 선분을 그리는 과정에서 각 선분의 중간점 좌표를 계산하고, 이 점들이 형상의 위쪽 경계(`top_surf_list`), 아래쪽 경계(`bot_surf_list`), 또는 바깥쪽 경계(`outer_surf_no_boundary_list`)에 속하는지 판단하여 각각의 리스트에 저장합니다. 이 정보는 나중에 경계 조건을 설정할 때 사용됩니다.

```
        permutations_np = np.array(list)



        cur_surf_list = []



        spline_list = []



        longest_permutation_flag = False

        if len(permutations_np)-1 > longest_permutation:

            longest_permutation = len(permutations_np)-1

            longest_permutation_flag = True

            outer_surf_no_boundary_list = []

        for i in range(len(permutations_np)-1):



            point1x = coords[permutations_np[i],0]

            point1y = coords[permutations_np[i],1]



            point2x = coords[permutations_np[i+1],0]

            point2y = coords[permutations_np[i+1],1]
```

```
            point1 = np.array([point1x,point1y])

            point2 = np.array([point2x,point2y])

            v_12 = point2 - point1



            midpoint = point1 + 0.5*v_12



            cur_surf_list.append(((midpoint[0], midpoint[1], 0.),))



            if idx == 0 and i==0:

                inner_node_list.append(((midpoint[0]+1.e-3, midpoint[1]+1.e-
3, 0.),))



            if np.isclose(midpoint[1], 1.):

                top_surf_list.append(((midpoint[0], midpoint[1], 0.),))

            elif np.isclose(midpoint[1], 0.):

                bot_surf_list.append(((midpoint[0], midpoint[1], 0.),))

            elif longest_permutation_flag:

                outer_surf_no_boundary_list.append(((midpoint[0],
midpoint[1], 0.),))



            if splined:

                # case distinguishment

                point1_boundary = False

                point2_boundary = False
```

```
                if idx == 0:
                    if ((np.isclose(point1x, 0.) or np.isclose(point1x, 1.)) or
                        (np.isclose(point1y, 0.) or np.isclose(point1y, 1.))):
                        point1_boundary = True

                    if ((np.isclose(point2x, 0.) or np.isclose(point2x, 1.)) or
                        (np.isclose(point2y, 0.) or np.isclose(point2y, 1.))):
                        point2_boundary = True

                    if point1_boundary and point2_boundary:
                        m.sketches['__profile__'].Line(point1=(point1[0], point1[1]),
                            point2=(point2[0], point2[1]))
                    elif point1_boundary and not point2_boundary:
                        spline_list.append(((point1[0],point1[1],)))
                    elif not point1_boundary and point2_boundary:
                        spline_list.append(((point2[0],point2[1],)))
                        m.sketches['__profile__'].Spline(points=(spline_list))
                        spline_list = []
                    elif not point1_boundary and not point2_boundary:
                        spline_list.append(((midpoint[0],midpoint[1],)))
                else:
                    if i == 0:
```

```
                        init_midpoint = midpoint

                    spline_list.append(((midpoint[0],midpoint[1],)))

                    if i == len(permutations_np)-2:

spline_list.append(((init_midpoint[0],init_midpoint[1],)))

                        m.sketches['__profile__'].Spline(points=
(spline_list))



            else:

                if radius > 0:

                    if i == len(permutations_np)-2:

                        point3x = coords[permutations_np[1],0]

                        point3y = coords[permutations_np[1],1]

                    else:

                        point3x = coords[permutations_np[i+2],0]

                        point3y = coords[permutations_np[i+2],1]

                    point3 = np.array([point3x,point3y])

                    v_23 = point3 - point2



                    if radius-1.e-5 > np.linalg.norm(v_12)/2 or radius-1.e-5
> np.linalg.norm(v_23)/2:

                        raise ValueError('Radius too large for given mesh.')



                    v_12_hat = v_12/np.linalg.norm(v_12)

                    v_23_hat = v_23/np.linalg.norm(v_23)
```

```
                    point1r = point1 + radius*v_12_hat

                    point2r = point2 - radius*v_12_hat

                    point3r = point2 + radius*v_23_hat


                    center = point2 - radius*v_12_hat + radius*v_23_hat


                    direction_vec = np.cross(-v_12, v_23)

                    if direction_vec > 0:

                        direction = CLOCKWISE

                    else:

                        direction = COUNTERCLOCKWISE


                    m.sketches['__profile__'].Line(point1=(point1r[0],
point1r[1]),

                        point2=(point2r[0], point2r[1]))


                    m.sketches['__profile__'].ArcByCenterEnds(center=
(center[0],center[1]),

                        direction=direction, point1=(point2r[0],
point2r[1]), point2=(point3r[0], point3r[1]))


                else:

                    m.sketches['__profile__'].Line(point1=(point1[0],
point1[1]),

                        point2=(point2[0], point2[1]))

        full_surf_list.append(cur_surf_list)
```

```
    m.Part(dimensionality=TWO_D_PLANAR, name=part_name, type=

        DEFORMABLE_BODY)

    m.parts[part_name].BaseShell(sketch=

        m.sketches['__profile__'])

    del m.sketches['__profile__']

  부품 생성: `m.Part`와 `BaseShell` 명령어를 사용하여 완성된 스케치로부터 2D 평면 부품(`Part`)을 생성합니다.

    return full_surf_list, top_surf_list, bot_surf_list, outer_surf_no_boundary_list, inner_node_list
```